Shade Tree Node

Smart-Contract Security Review

Internal audit by Florian Castet

target	`contracts/`
repository	`dmarzzz/shade-tree-node`
Commit	`5bed894`
Date	2026-09-04

Contents

1 Findings summary **2**

2 Findings **2**

2.1 High Risk . . . 2

2.1.1 Any member can self-slash to recover their bond instantly; the slash payout is redirectable . . . 2

2.1.2 A non-canonical commitment (`c + p`) registers an un-slashable, un-exitable but fully functional leaf . . . 3

2.1.3 A commitment of 0 collides with the tree's "never written" sentinel and permanently splits the on-chain root from every off-chain reconstruction . . . 4

2.1.4 The tier is declared, not proven: a tier-32 leaf registered at tier 8 pays a quarter of the bond and can never be slashed . . . 5

2.2 Medium Risk . . . 5

2.2.1 Exit and withdraw proofs are replayable after re-registration; the context lacks any per-registration binding . . . 6

2.3 Low Risk . . . 6

2.3.1 Exit/withdraw proof blobs expose the identity commitment, linking one identity's leaves across tiers and deployments . . . 7

2.3.2 Permissionless `register` lets anyone front-run a victim's commitment at a different tier, permanently binding it to the wrong recorded tier . . . 7

2.3.3 A slashed commitment can be re-inserted into the paid set although its secret is public, selling a leaf anyone can immediately zero . . . 8

1 Findings summary

ID	Severity	Finding
3.1.1	High	Any member can self-slash to recover their bond instantly; the slash payout is redirectable
3.1.2	High	A non-canonical commitment (`c + p`) registers an un-slashable, un-exitable but fully functional l...
3.1.3	High	A commitment of 0 collides with the tree's "never written" sentinel and permanently splits the on-chain...
3.1.4	High	The tier is declared, not proven: a tier-32 leaf registered at tier 8 pays a quarter of the bond and can ...
3.2.1	Medium	Exit and withdraw proofs are replayable after re-registration; the context lacks any per-registration bin...
3.3.1	Low	Exit/withdraw proof blobs expose the identity commitment, linking one identity's leaves across tiers and ...
3.3.2	Low	Permissionless `register` lets anyone front-run a victim's commitment at a different tier, permane...
3.3.3	Low	A slashed commitment can be re-inserted into the paid set although its secret is public, selling a leaf a...

2 Findings

2.1 High Risk

2.1.1 Any member can self-slash to recover their bond instantly; the slash payout is redirectable

Severity: High Risk

Context: `StakedReputationSet.sol` (`slash`)

Description. `slash(commitment, secret, limit, receiver)` is `public` and authorized solely by knowledge of the `(commitment, secret, limit)` triple — a triple the member *always* possesses, since they generated the secret. The full bond is paid to a caller-chosen `receiver`. Two consequences: (1) a member calls `slash` on their own leaf with `receiver = self` and receives the entire bond in one transaction, bypassing `initiateExit`, the `UNBONDING` time-lock and requirement R4 ("spam then instantly unstake is impossible") entirely; (2) when the gateway broadcasts a punitive slash carrying the reconstructed secret, the offender (or any searcher) front-runs it with their own `receiver`, so the punishment pays the offender and the gateway's transaction reverts `NotMember`. The economic deterrent of R3/R4 is void: over-spending never costs the bond. The documentation's "whoever reconstructed the secret claims the bond" and invariant I4 are inaccurate. **(`PaidAccessSet.slash` pays nothing and is unaffected.)**

Impact. A member can recover their entire bond at will (self-slash) and can also redirect an honest gateway's punitive slash to themselves, so over-spending never costs the bond — the R3/R4 economic deterrent is void.

Recommendation. Burn the slashed bond (send to a dead address) or pay a fixed protocol treasury, optionally granting `msg.sender` a small fixed bounty; never let the caller name the payout target. Commit–reveal or a private relay only fixes the reward race, not the self-slash refund. Update ONCHAIN.md R3/R4 and CONTRACTS-AUDIT.md I4 accordingly.

Proof of concept.

```
// A member can recover their entire bond at will by slashing their own leaf, with no exit and
// no unbonding wait; an honest gateway's later punitive slash of that leaf then reverts.
function test_memberSelfSlashRecoversFullBondBypassingUnbonding() public {
    StakedReputationSet set = _newSet(IWithdrawVerifier(address(mockVerifier)));
    address memberPayout = address(0xF00D); // a fresh address the member controls
    set.register{value: BOND}(cA);
    // The member never calls initiateExit and never waits UNBONDING. slash() is authorised only
    // by the secret they themselves generated, and pays the bond to a caller-chosen address.
    uint256 before = memberPayout.balance;
    set.slash(cA, SECRET_A, 8, memberPayout);
    assertEq(memberPayout.balance - before, BOND, "member recovered the full bond instantly via self-slash
     ");
    (uint256 bond,,,) = set.members(cA);
    assertEq(bond, 0, "leaf removed with no unbonding wait");
    // The offender's leaf is gone, so a gateway's punitive slash now reverts: the deterrent is void.
    vm.expectRevert(StakedReputationSet.NotMember.selector);
    set.slash(cA, SECRET_A, 8, address(0x6A7E));
}
```

2.1.2 A non-canonical commitment (`c + p`) registers an un-slashable, un-exitable but fully functional leaf

Severity: High Risk

Context: `StakedReputationSet.sol` / `PaidAccessSet.sol` (`register` / `_insert`); `PoseidonT3.sol`

Description. `commitment` is a raw `uint256`; nothing requires it to be a canonical BN254 field element. Both the on-chain Poseidon library and the off-chain `poseidon-lite` reduce inputs mod p, so registering $c + p$ (which fits in 256 bits for every field element c) inserts a tree leaf that hashes *identically* to the honest leaf c — RLN membership proofs verify against `currentRoot` — while the storage key is $c + p$. `slash(c, ...)` reverts `NotMember` (no record), `slash(c+p, ...)` reverts `BadSecret` (the hasher returns $c \neq c + p$), and exit/withdraw fail the leaf-reconstruction check the same way. The gateway's `limitOf(c)` probe finds nothing, and after a failed on-chain slash it applies no local ban, so the member over-spends every epoch with no on-chain consequence for a one-time bond. Registering both c and $c + p$ gives one identity two leaves; slashing c leaves the identity in the root. The paid set's registrar rejects $\geq p$ at the HTTP layer but the contract does not.

Impact. For a single bond a member obtains a permanent membership that can never be slashed, exited, or removed on chain, so that leaf's over-spend deterrent is gone for good.

Recommendation. Revert on non-canonical leaves in `register` and `_insert`: `if (commitment == 0 || commitment >= FIELD) revert BadCommitment();`. The complete fix is to derive the leaf on chain (see 3.1.4), which makes non-canonical keys unrepresentable. Add fuzz generators drawing commitments from the full `uint256` range.

Proof of concept.

```
// Registering a commitment >= the field prime yields a leaf that hashes identically to the
// canonical leaf (membership works) yet can never be slashed or exited (the key never matches).
function test_nonCanonicalCommitmentYieldsUnslashableLeaf() public {
    uint256 cAp = cA + P; // < 2^256, a distinct uint256 key
    assertEq(PoseidonT3.hash([cAp, 5]), PoseidonT3.hash([cA, 5]), "Poseidon reduces inputs mod p");
    StakedReputationSet set = _newSet(IWithdrawVerifier(address(realVerifier)));
    set.register{value: BOND}(cAp);
    uint256[] memory leaves = new uint256[](1); leaves[0] = cA;
    assertEq(set.currentRoot(), _refRoot(leaves), "the tree leaf equals the honest, canonical leaf cA");
    vm.expectRevert(StakedReputationSet.NotMember.selector);
    set.slash(cA, SECRET_A, 8, ATTACKER);
    vm.expectRevert(StakedReputationSet.BadSecret.selector);
    set.slash(cAp, SECRET_A, 8, ATTACKER);
    vm.expectRevert(StakedReputationSet.BadProof.selector);
    set.initiateExit(cAp, exitProof);
```

```
    assertEq(set.limitOf(cA), 0, "a gateway's limitOf(cA) lookup finds nothing to slash");
}
```

2.1.3 A commitment of 0 collides with the tree's "never written" sentinel and permanently splits the on-chain root from every off-chain reconstruction

Severity: High Risk

Context: `StakedReputationSet.sol` / `PaidAccessSet.sol` (`_nodeAt` sentinel)

Description. The sparse node store uses 0 to mean "never written" and substitutes the level's empty-subtree value on read (`_nodeAt`). The stated justification — "real leaves are Poseidon field elements, so 0 unambiguously means never written" — holds for internal nodes but not for leaves: `register(0, 8)` is accepted from anyone for one bond, and the paid-set registrar's commitment regex explicitly admits `"0"`. When leaf 0 is written both sides agree; on the very next update touching its sibling, the contract reads leaf 0 back as `_zeroes[0]` while every event-replaying reader (zk-kit IMT 0.4.3 has no zero-leaf check) keeps the literal 0. From then on `currentRoot` differs permanently from every off-chain reconstruction: the light-client path (`eth_getProof` of slot 3 — the trust-minimized reading T-DEV-9 exists for) proves a root that no client's proofs match, and no runtime code compares the two sources, so the split is silent. One bond disables the light-client provider for the whole set.

Impact. One 0-commitment registration permanently de-synchronises the on-chain root from every off-chain reconstruction, bricking the trust-minimised light-client root path for the entire set.

Recommendation. Reject `commitment == 0` (subsumed by the 3.1.2 check or by on-chain leaf derivation, 3.1.4). Independently, make the sentinel unambiguous (store `value + 1` or an explicit written-flag), and add a runtime consistency check in `lib/root-provider.mjs` comparing the reconstructed root with `currentRoot()`, failing loudly on mismatch.

Proof of concept.

```
// A commitment of 0 collides with the tree's "unset" sentinel, so the on-chain root
// permanently diverges from any off-chain reconstruction over the same leaves.
function test_zeroCommitmentDesyncsOnchainRootFromReconstruction() public {
    StakedReputationSet set = _newSet(IWithdrawVerifier(address(mockVerifier)));
    set.register{value: BOND}(0);
    set.register{value: BOND}(cA);
    uint256[] memory leaves = new uint256[](2); leaves[0] = 0; leaves[1] = cA;
    assertTrue(set.currentRoot() != _refRoot(leaves), "on-chain root != reconstruction over [0, cA]");
    leaves[0] = zeroes[0];
    assertEq(set.currentRoot(), _refRoot(leaves), "on-chain root == a tree where leaf 0 is treated as EMPTY
    ");
    // Same divergence on the paid set (operator/buyer-supplied commitment).
    uint256[] memory lim = new uint256[](1); lim[0] = 8;
    PaidAccessSet pas = new PaidAccessSet(address(this), ICommitmentHasher(address(hasher)), lim);
    pas.insert(0, 8);
    pas.insert(cA, 8);
    assertEq(pas.currentRoot(), set.currentRoot(), "paid set exhibits the identical divergence");
}
```

2.1.4 The tier is declared, not proven: a tier-32 leaf registered at tier 8 pays a quarter of the bond and can never be slashed

Severity: High Risk

Context: `StakedReputationSet.sol` / `PaidAccessSet.sol` (`register` / `insert`)

Description. The leaf commits to its own tier (Poseidon2(Poseidon1(s), *limit*)), but `register(commitment, limit)` prices and records whatever `limit` the caller names — the contract cannot look inside the leaf. A leaf built at limit 32 registered as limit 8 (i) pays `BOND` instead of 4×`BOND` while the RLN circuit grants the leaf's real 32-message budget, and (ii) is permanently immune to on-chain removal: `slash(c, s, 8)` fails `BadSecret`, `slash(c, s, 32)` fails `BadLimit`, and exit/withdraw fail the recorded-tier leaf check. The documentation acknowledges the lock but claims the mismatch "buys nothing"; it buys a four-fold budget discount and immunity from slashing (combined with the absence of any gateway-side local ban). The same declaration gap exists in `PaidAccessSet.insert`, where the registrar copies the buyer's declared tier.

Impact. A member obtains a higher tier's message budget for the cheaper tier's bond and a leaf that can never be slashed on chain, at will.

Recommendation. Derive the leaf on chain: change the entry points to `register(uint256 identityCommitment, uint256 limit)` / `insert(...)` and compute `commitment = PoseidonT3.hash([iden limit])` (one Poseidon, ≈60k gas, library already linked). The identity commitment reveals nothing beyond the public leaf. This single change makes the tier provable and every stored leaf canonical and non-zero, closing 3.1.2 and 3.1.3 as well, and lets `slash` drop its `limit` argument. Because this changes the `register`/`insert` ABI, it is a cross-cutting change: every off-chain caller that produces or submits a commitment — the enrollment and payment tooling, the paid-access registrar, the browser staking page, and the Rust client — must move to the identity-commitment input at the same time.

Proof of concept.

```
// A leaf built at a high tier but declared at the cheap tier is admitted for the cheap bond and
// is permanently un-slashable and un-exitable.
function test_declaredTierMismatchBuysCheapUnslashableHighTierLeaf() public {
    StakedReputationSet set = _newSet(IWithdrawVerifier(address(mockVerifier)));
    uint256 c32 = hasher.commitmentOf(SECRET_A, 32); // a genuine tier-32 leaf
    set.register{value: BOND}(c32, 8);               // pays 1x BOND, not 4x
    assertEq(set.limitOf(c32), 8, "recorded as tier 8");
    uint256[] memory leaves = new uint256[](1); leaves[0] = c32;
    assertEq(set.currentRoot(), _refRoot(leaves), "the tier-32 leaf is admitted into the root");
    vm.expectRevert(StakedReputationSet.BadSecret.selector);
    set.slash(c32, SECRET_A, 8, ATTACKER);
    vm.expectRevert(StakedReputationSet.BadLimit.selector);
    set.slash(c32, SECRET_A, 32, ATTACKER);
    vm.expectRevert(StakedReputationSet.BadProof.selector);
    set.initiateExit(c32, abi.encode(SECRET_A));
}
```

2.2 Medium Risk

2.2.1 Exit and withdraw proofs are replayable after re-registration; the context lacks any per-registration binding

Severity: Medium Risk

Context: `StakedReputationSet.sol` (exit/withdraw contexts); `WithdrawVerifier.sol`

Description. The exit context is `keccak256("SHADE_TREE_EXIT", commitment)` and the withdraw context `keccak256("SHADE_TREE_WITHDRAW", commitment, recipient)`. Neither includes anything that changes between two registrations of the same commitment; the withdraw circuit has no nullifier and the verifier is stateless, so replay protection is purely contract state — which is erased when a withdrawn or slashed commitment is deleted and (by documented design) re-registerable. After a member's honest register–exit–withdraw cycle, both proofs are public calldata. Once the same commitment is registered again (by the member, or by anyone — registration is permissionless), any third party replays the old exit proof to force the member out of the admission root, then after `UNBONDING` replays the old withdraw proof, sending the new bond to the *old* recipient. The victim cannot prevent it, loses the fresh unlinkable-recipient property (R2), and loses the funds if the old recipient is stale (relayer, exchange deposit address).

Impact. After a commitment is re-registered, any third party (no secret, only gas) can force the member out of the admission root and redirect the new bond to the member's stale first-cycle recipient.

Recommendation. Bind each registration instance into the context, e.g. `keccak256(tag, commitment, m.index[, recipient])` — the leaf index is unique per registration and already stored. Regenerate `testdata/withdraw-proof.json` and update the docs, which currently call re-registration "harmless" and describe a non-existent "withdraw-scoped nullifier".

Proof of concept.

```
// An exit/withdraw proof stays valid after the commitment is re-registered, so a third party
// (no secret) can force the member out and pay the new bond to the member's stale recipient.
function test_exitAndWithdrawProofsReplayableAfterReRegistration() public {
    StakedReputationSet set = _newSet(IWithdrawVerifier(address(realVerifier)));
    set.register{value: BOND}(cA);
    set.initiateExit(cA, exitProof);
    vm.warp(block.timestamp + UNBONDING);
    set.withdraw(cA, RECIPIENT, withdrawProof); // honest cycle; both proofs now public
    set.register{value: BOND}(cA);              // re-registration (permitted)
    assertTrue(set.isActive(cA));
    vm.prank(ATTACKER);
    set.initiateExit(cA, exitProof);            // replayed exit proof accepted
    assertFalse(set.isActive(cA));
    vm.warp(block.timestamp + UNBONDING);
    uint256 before = RECIPIENT.balance;
    vm.prank(ATTACKER);
    set.withdraw(cA, RECIPIENT, withdrawProof);  // replayed withdraw proof pays the stale recipient
    assertEq(RECIPIENT.balance - before, BOND, "new bond forced out to the old recipient");
}
```

2.3 Low Risk

2.3.1 Exit/withdraw proof blobs expose the identity commitment, linking one identity's leaves across tiers and deployments

Severity: Low Risk

Context: `WithdrawVerifier.sol` (the proof's public `identityCommitment`)

Description. The proof encoding carries `identityCommitment` as a public value, and the leaf formula Poseidon2($idC, limit$) is public, so one exit lets any observer recompute that identity's leaf at every admitted tier and on every deployment, linking memberships that were meant to be unlinkable. This is inherent to the wrapper's design (the circuit outputs `idC`); the privacy loss should at least be documented, and it compounds 3.3.3 by making revoked identities recognizable.

Impact. Once a member exits, an observer can recompute and link that identity's leaves across every tier and deployment (identity linkage, not traffic content).

Recommendation. Document the linkage; longer-term, use a withdraw circuit that takes the rate-commitment leaf (or a nullifier) as its public output instead of the raw identity commitment.

Proof of concept.

```
// The exit/withdraw proof publishes identityCommitment, and the leaf formula is public, so one
// exit lets any observer recompute (link) that identity's leaves at every tier.
function test_exitProofIdentityCommitmentLinksLeavesAcrossTiers() public view {
    uint256 idc = PoseidonT2.hash([SECRET_A]); // the circuit's public output
    uint256 leaf8 = PoseidonT3.hash([idc, 8]);
    uint256 leaf32 = PoseidonT3.hash([idc, 32]);
    assertEq(leaf8, hasher.commitmentOf(SECRET_A, 8), "tier-8 leaf recomputed from the public identity
     commitment");
    assertEq(leaf32, hasher.commitmentOf(SECRET_A, 32), "tier-32 leaf recomputed from the same identity
     commitment");
    assertTrue(leaf8 != leaf32, "distinct leaves, one identity, both derivable by any observer");
}
```

2.3.2 Permissionless `register` lets anyone front-run a victim's commitment at a different tier, permanently binding it to the wrong recorded tier

Severity: Low Risk

Context: `StakedReputationSet.sol` (`register`)

Description. Registration requires no proof of knowledge of the secret and keys the record by the raw commitment. An attacker who sees a victim's commitment (mempool or prior events) can register it first at another tier, paying that tier's bond. The victim's own registration then reverts `AlreadyMember` for ever, and the leaf — recorded at a tier it was not derived at — is un-exitable and un-slashable (3.1.4 mechanics). The victim's identity must be abandoned; the attacker's cost is one bond per grief. On-chain leaf derivation (3.1.4's fix) removes the wrong-tier variant; a proof-of-knowledge at registration would remove occupation entirely.

Impact. An attacker can permanently lock a victim's chosen commitment at a wrong tier for the price of one bond; pure griefing with no attacker gain (subsumed by the 3.1.4 fix).

Recommendation. Fix via 3.1.4's on-chain leaf derivation; if commitment occupation by third parties matters, additionally require a signature/proof binding the registration to the secret holder.

Proof of concept.

```
// Registration needs no proof of secret knowledge, so an attacker can front-run a victim's
// commitment at a wrong tier, permanently locking it (un-registerable, un-slashable, un-exitable).
```

```
function test_attackerFrontRunsRegistrationLockingCommitmentAtWrongTier() public {
    StakedReputationSet set = _newSet(IWithdrawVerifier(address(mockVerifier)));
    uint256 victimLeaf = hasher.commitmentOf(SECRET_A, 32); // the victim's real tier-32 leaf
    vm.prank(ATTACKER);
    set.register{value: BOND}(victimLeaf, 8); // attacker registers it as tier 8 for the cheap bond
    vm.expectRevert(StakedReputationSet.AlreadyMember.selector);
    set.register{value: 4 * BOND}(victimLeaf, 32); // the victim can no longer register their own leaf
    vm.expectRevert(StakedReputationSet.BadSecret.selector);
    set.slash(victimLeaf, SECRET_A, 8, ATTACKER);
    vm.expectRevert(StakedReputationSet.BadLimit.selector);
    set.slash(victimLeaf, SECRET_A, 32, ATTACKER);
}
```

2.3.3 A slashed commitment can be re-inserted into the paid set although its secret is public, selling a leaf anyone can immediately zero

Severity: Low Risk

Context: `PaidAccessSet.sol` (`_insert` / `slash`)

Description. `_insert` rejects only a *currently live* commitment. A slash publishes the identity secret in calldata forever, so a buyer who re-purchases with the same identity receives a leaf that any observer of the old slash transaction can permanently zero with a one-line call — no bond, no cooldown, no refund. The staked set documents the equivalent re-registration as harmless ("the secret is already public"), which is exactly why the paid product should refuse it.

Impact. A buyer who voluntarily reuses a slashed identity's (now public) secret buys a paid leaf that anyone can immediately zero; only affects deliberate secret reuse.

Recommendation. Keep a burned-commitment set (or check historical `Slashed` state) and reject re-insertion; at minimum, make the registrar warn the buyer and refuse to sell a leaf for a commitment with a public secret. *Fixed at commit `082fa8d`.*

Proof of concept.

```
// A slashed paid commitment can be inserted again, but its secret is already public, so the
// re-purchased leaf can be zeroed immediately by anyone.
function test_slashedPaidCommitmentReinsertableThenZeroableByAnyone() public {
    uint256[] memory lim = new uint256[](1); lim[0] = 8;
    PaidAccessSet pas = new PaidAccessSet(address(this), ICommitmentHasher(address(hasher)), lim);
    uint256 leaf = hasher.commitmentOf(SECRET_A, 8);
    pas.insert(leaf, 8);
    pas.slash(leaf, SECRET_A, 8, address(0)); // over-spend reveals SECRET_A; leaf zeroed
    assertEq(pas.limitOf(leaf), 0, "leaf slashed");
    pas.insert(leaf, 8); // a buyer reusing the same identity re-purchases: re-insert is allowed
    assertEq(pas.limitOf(leaf), 8, "same commitment re-inserted after a slash");
    vm.prank(ATTACKER);
    pas.slash(leaf, SECRET_A, 8, address(0)); // the public secret lets anyone zero it again
    assertEq(pas.limitOf(leaf), 0, "re-purchased leaf instantly zeroed by a third party");
}
```